



INTERNET PROGRAMMING AND MOBILE PROGRAMMING

TASK 7: FINAL REPORT

PROJECT 3: Design and Implementation of a Crowdsensed Drive Test Mobile Application for Coverage Prediction

PRESENTED BY

GROUP 10

GROUP MEMBERS

	Name Of Member	Matricule
1	Asangana Paul Fleming	FE23A014
2	Fontem Junior Nkonganyik	FE23A063
3	Suinyuy Mathias Mahla	FE23A144
4	Nfor Godwill Kongnje	FE23A108
5	Ronald Namwene Vefonge	FE22A295

INSTRUCTOR: Dr. Nkemeni Valery

TASK 6: DATABASE DESIGN AND IMPLEMENTATION	1
CHAPTER 1: GENERAL INTRODUCTION.....	3
1.1 Background and Context of the Study.....	3
1.2 Problem statement	5
1.3. Objectives of the Study	5
1.3.1. General Objective	5
1.3.2. Specific Objectives	5
1.4. Proposed Methodology	5
1.5. Research Questions (if applicable)	6
1.6. Research Hypothesis (if applicable).....	6
CHAPTER 2: LITERATURE REVIEW.....	8
2.1. Introduction	8
2.2. General Concepts	8
2.3. Related Works	11
2.4. Partial Conclusion.....	12
CHAPTER 3: ANALYSIS AND DESIGN.....	13
3.1. Introduction	13
3.2. ProposedMethodology	13
3.3. Design	14
3.4. Global Architecture of the solution	20
3.5. Description of the resolution process	20
3.6. Partial conclusion	20
CHAPTER 4: IMPLEMENTATION (or REALIZATION) AND RESULTS	
4.1 Introduction	21
4.2 Database Architecture and Conceptual Design.....	22
4.3 Ingestion Pipeline Implementation.....	25
4.4 Data Quality Categorization and Analysis	26
4.5. Partial conclusion	27
CHAPTER 5: CONCLUSION.....	28
5.1. Summary of findings	28
5.2. Contribution to engineering and technology	28
5.3. Recommendations	29
5.4. Difficulties encountered	29
5.5. Further works	29

CHAPTER 1: GENERAL INTRODUCTION

1.1 Background and Context of the Study

The rapid growth of mobile communication technologies has transformed the way individuals, businesses, and governments communicate and access digital services. Mobile networks have become an essential component of modern society, supporting services such as voice communication, video conferencing, online education, mobile banking, electronic commerce, emergency response, healthcare services, and Internet of Things (IoT) applications. As the demand for reliable mobile connectivity continues to increase, mobile network operators are expected to provide high-quality network coverage and uninterrupted communication services to users regardless of their location.

One of the major challenges faced by mobile network operators is maintaining consistent network coverage across different geographical locations. Although significant investments have been made in deploying cellular infrastructure such as base stations and communication towers, many areas still experience poor signal strength, unstable connections, dropped calls, slow internet speeds, and network congestion. These areas are commonly referred to as **coverage holes** or **coverage blind spots**. Coverage holes negatively affect user satisfaction, business productivity, emergency communications, and the overall Quality of Service (QoS) provided by network operators.

To identify these problem areas, network operators traditionally conduct **drive tests**, where specialized vehicles equipped with expensive radio frequency (RF) measurement equipment travel along predefined routes while collecting information about network performance. During these drive tests, engineers collect measurements such as Reference Signal Received Power (RSRP), Reference Signal Received Quality (RSRQ), Received Signal Strength Indicator (RSSI), Signal-to-Interference-plus-Noise Ratio (SINR), throughput, Cell ID, and geographical coordinates.

Although traditional drive testing provides accurate results, it has several limitations. It requires costly equipment, trained personnel, dedicated vehicles, fuel, and significant time to complete. Since drive tests are performed only occasionally, they provide limited information about changing network conditions. They are also difficult to conduct in rural communities, mountainous regions, forests, and other difficult-to-access locations. As mobile networks continue expanding and user demands increase, traditional drive testing alone is no longer sufficient for continuous network monitoring.

Recent advancements in smartphone technology have introduced an alternative approach known as **mobile crowdsensing**. Modern smartphones contain numerous built-in sensors, including GPS receivers, cellular radio modules, accelerometers, gyroscopes, and internet connectivity. These capabilities allow smartphones to function as intelligent sensing devices capable of collecting valuable network information while users carry out their normal daily activities.

Mobile crowdsensing refers to the process of collecting data from many smartphones distributed across different locations. Instead of relying on a few engineers conducting periodic drive tests, thousands of smartphone users can continuously contribute

network measurements. Each smartphone records signal strength, network quality, serving cell information, GPS location, and timestamps. These measurements are then transmitted to a centralized backend server where they are aggregated, analyzed, and visualized to provide a comprehensive representation of network coverage.

The availability of cloud computing, high-speed internet, mobile application development frameworks, and machine learning techniques has further improved the effectiveness of crowdsensed network monitoring systems. Large volumes of measurements collected from numerous users can be processed efficiently to identify coverage holes, evaluate network quality, and even predict areas likely to experience poor coverage before customer complaints are received.

This project focuses on designing and implementing a Crowdsensed Drive Test Mobile Application for Coverage Prediction. The proposed system will use Android smartphones as Class-2 Internet of Things (IoT) devices to automatically collect cellular network measurements together with geographical information. The collected measurements will be stored locally whenever internet connectivity is unavailable and synchronized with a backend server once connectivity is restored. The backend platform will process the received measurements, visualize network coverage, identify coverage holes, and support future coverage prediction.

The implementation of such a system will provide a cost-effective, scalable, and continuously operating alternative to traditional drive testing while improving network monitoring and decision-making for mobile network operators.

1.2 Problem Statement

Despite the continuous expansion of mobile communication infrastructure, many users still experience poor network quality, weak signal strength, dropped calls, and slow internet connectivity in various locations. Mobile network operators currently depend heavily on traditional drive testing techniques to assess network performance and identify coverage problems. While these methods provide accurate measurements, they require expensive equipment, skilled personnel, dedicated vehicles, and considerable operational costs.

Traditional drive tests are conducted periodically rather than continuously, making it difficult to monitor rapidly changing network conditions. Furthermore, large geographical regions, especially rural and difficult-to-access areas, may not be adequately covered due to financial and logistical constraints. Consequently, many coverage holes remain undetected until customers report network problems.

Modern smartphones possess advanced sensing capabilities that can continuously collect network measurements while users perform their normal daily activities. However, many existing systems do not effectively utilize this distributed sensing capability for large-scale network monitoring and coverage prediction.

There is therefore a need to develop an intelligent crowdsensed drive test mobile application capable of collecting real-time cellular network measurements from multiple smartphones, securely transmitting the data to a centralized backend platform, and analyzing the collected information to identify and predict mobile network coverage holes at a significantly lower cost than conventional drive testing methods.

1.3 Objectives of the Study

1.3.1 General Objective

The general objective of this study is to design, implement, and evaluate a crowdsensed drive test mobile application that utilizes smartphones as sensing devices to collect cellular network measurements for mobile network coverage prediction and coverage hole detection.

1.3.2 Specific Objectives

The specific objectives are to:

- Design a mobile crowdsensing architecture for collecting cellular network measurements using smartphones.
- Develop an Android mobile application capable of acquiring network parameters such as RSRP, RSRQ, RSSI, SINR, Cell ID, GPS coordinates, and timestamps.
- Implement a secure mechanism for storing collected measurements locally during offline operation.
- Develop a backend server for receiving, storing, and managing crowdsensed network measurements.
- Implement data synchronization between the mobile application and the backend server whenever internet connectivity becomes available.
- Develop algorithms for analyzing collected measurements to identify network coverage holes.
- Visualize collected network measurements using geographical maps.
- Evaluate the performance of the developed system under different network conditions.

1.4 Proposed Methodology

- ❖ The study will adopt the Design Science Research methodology combined with an Agile software development approach.
- ❖ The project began by conducting a literature review to understand existing drive testing techniques, mobile crowdsensing technologies, smartphone sensing capabilities, and network coverage prediction methods. System requirements will then be gathered and analyzed to identify both functional and non-functional requirements.
- ❖ Based on these requirements, the overall system architecture will be designed, consisting of an Android mobile application, a backend server, a centralized database, and a web-based monitoring dashboard.
- ❖ The Android application will be developed to collect network measurements together with GPS coordinates and timestamps. The application will also implement offline data storage using a local database to ensure measurements are not lost when internet connectivity is unavailable.
- ❖ A backend server will be developed to receive uploaded measurements through secure RESTful APIs. Received measurements will be stored in a centralized database and processed using statistical analysis and prediction algorithms to

identify coverage holes and estimate network quality in different geographical regions.

- ❖ The completed system will undergo unit testing, integration testing, system testing, and user acceptance testing to verify that all functional requirements have been satisfied.

1.5 Research Questions

The study seeks to answer the following research questions:

1. How can smartphones be effectively used as mobile sensing devices for cellular network monitoring?
2. How can crowdsensed measurements improve traditional drive testing methods?
3. What techniques can be used to securely collect, store, and transmit mobile network measurements?
4. How accurately can crowdsensed measurements identify network coverage holes?
5. Can crowdsensed measurements be used to predict mobile network coverage in unmeasured areas?

1.6 Research Hypothesis

- Null Hypothesis (H_0): A crowdsensed drive test mobile application does not significantly improve the efficiency of mobile network coverage monitoring compared to traditional drive testing.
- Alternative Hypothesis (H_1): A crowdsensed drive test mobile application significantly improves the efficiency, scalability, and cost-effectiveness of mobile network coverage monitoring compared to traditional drive testing.

1.7 Significance of the Study

This study is significant because it provides a practical and low-cost approach to monitoring mobile network coverage using smartphones instead of expensive drive test equipment. Mobile network operators will benefit from continuous and large-scale network measurements collected from real users. Researchers will gain additional knowledge in mobile crowdsensing, IoT, and wireless communication systems. The developed system can also support government regulators, telecommunications authorities, and service providers in making informed decisions regarding network expansion and quality improvement. Ultimately, mobile subscribers will benefit from improved network reliability and better communication services.

1.8 Scope of the Study

This study focuses on the design and implementation of an Android-based crowdsensed drive test mobile application for collecting cellular network measurements. The system will collect network parameters, GPS coordinates, timestamps, and device information, store measurements locally during offline operation, synchronize data with a centralized backend server, analyze network performance, identify coverage holes, and visualize network coverage. The study does not include direct control or optimization of mobile network infrastructure.

1.9 Delimitation of the Study

The study is limited to Android smartphones due to their broader access to cellular network APIs. Data collection depends on user participation and the permissions granted by device owners. Coverage prediction will be based only on the measurements collected by participating users, and the system will not interface directly with mobile network operators' proprietary infrastructure.

1.10 Definition of Keywords and Terms

- ❖ **Crowdsensing:** The collection of data from multiple users' smartphones to monitor or analyze a particular environment or system.
- ❖ **Drive Test:** A process of measuring the performance of a mobile network using specialized equipment installed in moving vehicles.
- ❖ **Coverage Hole:** A geographical area where mobile network coverage is weak or unavailable.
- ❖ **RSRP (Reference Signal Received Power):** A measure of the strength of the LTE reference signal received by a mobile device.
- ❖ **RSRQ (Reference Signal Received Quality):** A measurement indicating the quality of the received LTE signal.
- ❖ **RSSI (Received Signal Strength Indicator):** A measurement representing the total received signal power.
- ❖ **SINR (Signal-to-Interference-plus-Noise Ratio):** A measure of signal quality considering interference and background noise.
- ❖ **Cell ID:** A unique identifier assigned to a cellular base station serving a mobile device.
- ❖ **IoT (Internet of Things):** A network of connected devices capable of collecting and exchanging data.
- ❖ **Coverage Prediction:** The estimation of network quality in geographical locations using collected measurements and analytical techniques.

1.11 Organization of the Dissertation

This dissertation is organized into five chapters. Chapter One introduces the study by presenting the background, problem statement, objectives, methodology, research questions, significance, scope, delimitations, key terms, and organization of the dissertation. Chapter Two reviews relevant literature on mobile communication networks, drive testing, mobile crowdsensing, IoT technologies, and coverage prediction techniques. Chapter Three describes the research methodology, system design, development tools, and implementation procedures. Chapter Four presents the implementation results, system testing, performance evaluation, and discussion of findings. Finally, Chapter Five summarizes the study, presents the conclusions, outlines the limitations, and provides recommendations for future research and improvements.

CHAPTER 2: LITERATURE REVIEW

2.1. Introduction

The rapid proliferation of cellular network technologies and the exponential increase in mobile data consumption have made network coverage optimization a core priority for telecommunication service providers and regulatory bodies. Traditionally, assessing wireless network Quality of Service (QoS) and identifying coverage holes has relied heavily on conventional drive testing. This traditional approach involves using specialized, high-cost telemetry hardware, dedicated measurement software, and test vehicles driven across predetermined routes by specialized technical personnel. While highly accurate, traditional drive testing presents significant bottlenecks, including high operational expenses, limited geographical scalability, and an inability to capture real-time, indoor, or dynamic user-perceived network conditions.

To address these limitations, mobile crowdsensing (MCS) has emerged as a disruptive, cost-effective alternative. By leveraging the built-in sensors, telephony application programming interfaces (APIs), and global positioning systems (GPS) embedded within ordinary citizen smartphones, crowdsensed drive testing democratizes network monitoring. It transforms everyday user devices into autonomous sensing nodes that gather radio frequency (RF) metrics during routine daily mobility.

The purpose of this literature review is to examine the foundational theories, technological frameworks, and empirical studies underpinning crowdsensed mobile drive testing and cellular coverage prediction. This chapter provides a rigorous breakdown of key network metrics, evaluates existing platforms, explores prediction models, and synthesizes the technical contributions and limitations of contemporary works in this domain, establishing a clear academic and engineering justification for the development of the **SignalSense** application.

2.2. General Concepts on Crowdsensed Cellular Monitoring

To comprehensively understand the operational mechanics of the **SignalSense** platform, it is critical to explore the fundamental technological concepts, cellular radio frequency parameters, and architectural paradigms that define modern crowdsensed network testing.

2.2.1 Mobile Crowdsensing (MCS) Architecture

Mobile Crowdsensing is an architectural paradigm where a large, distributed group of individuals utilizes internet-connected mobile devices to collect and share spatio-temporal data to measure, analyze, or predict phenomena of common interest. Within the framework of telecommunication monitoring, MCS operates via an implicit or explicit participation model:

Implicit (Passive) Crowdsensing: The data collection software executes as a silent background service on the mobile device without requiring active user intervention or distracting from regular device usage. **SignalSense** leverages this passive approach to seamlessly log cellular quality variables without imposing operational cognitive load on the user.

Explicit (Active) Crowdsensing: The user manually triggers specific actions, such as executing an on-demand network speed test or actively reporting a localized "dead zone."

2.2.2 Key Cellular Radio Frequency (RF) Parameters

Evaluating cellular coverage requires gathering specific quantitative physical-layer metrics through smartphone Telephony APIs. The most critical data elements collected for analysis include:

RSSI (Received Signal Strength Indicator): A wideband power metric representing the total received power observed by the mobile device, including the target reference signals, co-channel interference, thermal noise, and multi-path propagation effects. Measured in dBm.

RSRP (Reference Signal Received Power): A highly precise linear power metric that measures the average power of the specific reference signals distributed across the cellular channel bandwidth. RSRP is the primary metric utilized in modern networks (4G LTE and 5G) to determine cell coverage boundaries and trigger handovers.

RSRQ (Reference Signal Received Quality): A dimensionless ratio that provides an index of signal quality.

SINR (Signal-to-Interference-plus-Noise Ratio): The ratio of the target signal power to the combined power of all co-channel interference signals and background thermal noise. It serves as a direct indicator of achievable data throughput and spectral efficiency.

Cell ID: A unique numerical identifier assigned to each base transceiver station (BTS) or sector, allowing crowdsensed measurements to be mapped back to specific physical infrastructure nodes.

2.3.3 Geographical and Metadata Elements

Radio measurements are meaningless without spatial and technical context. Accurate coverage prediction relies on binding RF parameters to precise spatio-temporal metadata:

Latitude and Longitude: Acquired via device Location APIs, defining the precise two-dimensional coordinate space of the measurement node.

Altitude: Crucial for mapping signal behavior across three-dimensional spatial topographies and mountainous terrains.

Speed: Derived from successive GPS samples to categorize whether data collection occurred under stationary, pedestrian, or vehicular conditions, which directly impact multi-path fading and signal stability.

GPS Accuracy: A filter metric used during data cleaning to reject low-precision coordinates caused by urban canyons or indoor attenuation.

2.3. Related Works

To contextualize the development of the **SignalSense** application, this section reviews established commercial platforms and academic methodologies used for crowdsensed network tracking, highlighting their core features, results, and structural limitations.

2.3.1 Review of Existing Platforms

The field of crowdsourced network quality analysis features a few prominent applications that have successfully implemented large-scale telemetry collection:

1. OpenSignal

OpenSignal is a commercial application that maps wireless coverage and tests network performance globally by gathering passive background metrics and active user speed tests.

Contributions: Successfully proved that massive, decentralized user bases could construct high-fidelity global coverage maps. They pioneered the use of consumer-grade smartphones to evaluate operator performance indicators objectively.

Limitations: The underlying data processing architectures, filtering algorithms, and prediction models are entirely proprietary. Furthermore, commercial constraints limit direct, unaggregated raw dataset access for independent researchers or regional network operators seeking localized deep-dives.

2. nPerf

nPerf focuses on comprehensive user-initiated performance testing, allowing mobile subscribers to test download speeds, upload speeds, latency, streaming stability, and browsing quality.

Contributions: Created a robust framework for multi-metric QoS evaluations, gathering end-to-end network performance metrics rather than just basic physical layer signal bars.

Limitations: Heavily reliant on active user engagement (explicit sensing). If a user does not manually run a test, coverage gaps or low-traffic zones remain completely unmonitored, resulting in sparse geographical data distribution.

3.2 Limitations Identified in Existing Literature

A systematic evaluation of current tools reveals several major technical gaps that the **SignalSense** project aims to resolve:

1. **High Battery Drain and Resource Consumption:** Continuous, high-frequency background polling of location and telephony hardware aggressively exhausts mobile battery lifespans. Existing literature often overlooks adaptive sampling loops, causing users to uninstall monitoring apps.

2. **Lack of Robust Offline Caching:** In severe coverage holes or rural regions, internet connectivity is entirely absent. Many legacy frameworks fail to gracefully manage state, discarding measurements when a real-time server upload fails.

3. Privacy Concerns and Forced Registration: A significant portion of modern telemetry platforms forces users to create accounts, link personal emails, or provide device identifiers that compromise privacy. This acts as a barrier to user participation, directly limiting data collection scale.

2.4. Partial Conclusion

The review of the literature demonstrates that while mobile crowdsensing provides a highly scalable, economically viable, and modern alternative to traditional vehicle-based drive testing, existing systems still suffer from structural deficiencies. These weaknesses include high battery consumption, strict account registration demands that compromise anonymity, and inadequate client-side caching mechanisms during prolonged network disconnections.

The **SignalSense** application addresses these gaps directly by establishing a lightweight, privacy-centric framework. By implementing an autonomous background sensing service optimized for minimal energy consumption, utilizing an on-device SQLite database container for resilient offline buffering, and eliminating registration barriers, **SignalSense** balances user privacy with data collection efficiency. The processed telemetry data is securely synchronized with a Node.js and MySQL backend infrastructure, establishing a clear path for accurate cellular coverage prediction, spatial heat map rendering, and the reliable detection of regional network dead zones.

CHAPTER 3: ANALYSIS AND DESIGN

3.1. Introduction

The primary objective of this project is to develop **SignalSense**, a crowdsensed drive test mobile application designed for cellular network coverage prediction and coverage hole detection. Traditional drive testing relies on expensive, specialized hardware mounted on dedicated vehicles, making the process logistically complex and difficult to scale. By leveraging the ubiquity of ordinary smartphones as crowdsensing nodes, this system collects real-time network parameters as users go about their daily routines.

This chapter outlines the structured framework used to analyze stakeholder expectations and translate them into robust system models. It encompasses data flow paths, object relationships, behavioral sequences, and the global deployment architecture necessary to build a highly available, fault-tolerant network monitoring platform.

3.2. Proposed Methodology

To address the limitations of traditional network mapping, this project adopts a **User-Centered Crowdsensing Methodology** engineered across distinct, iterative phases:

Requirement Gathering & Survey Sanitization: Utilizing structured feedback loop tools (such as Google Forms) to collect user behavioral trends, device ecosystems, and primary network pain points.

Iterative Software Requirement Specification (SRS): Transforming raw user data into formal functional requirements (measurement collection, offline caching, secure sync) and non-functional requirements (battery optimization, data privacy, scalability).

Object-Oriented System Modeling: Using Unified Modeling Language (UML) and Structured Analysis tools to build structural blueprints (Class Diagrams) and behavioral flows (DFDs, Use Cases, Sequence Diagrams).

Offline-First Architectural Design: Developing a hybrid data architecture utilizing lightweight local databases (**SQLite**) paired with a high-throughput backend server ecosystem (**Node.js** and **MySQL**).

3.3. Design

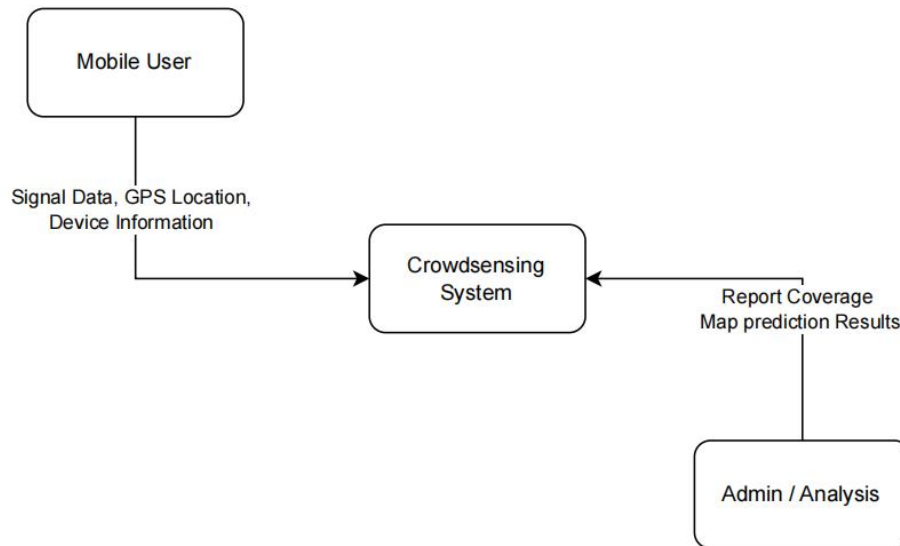
The system design establishes the logical rules governing how data moves through components, how objects are structured, and how stakeholders interact with the application boundary.

3.3.1 Data Flow Architectural Design (DFD)

The movement of technical radio metrics through the SignalSense platform is systematically decomposed into three operational levels:

Level 0: Context Diagram

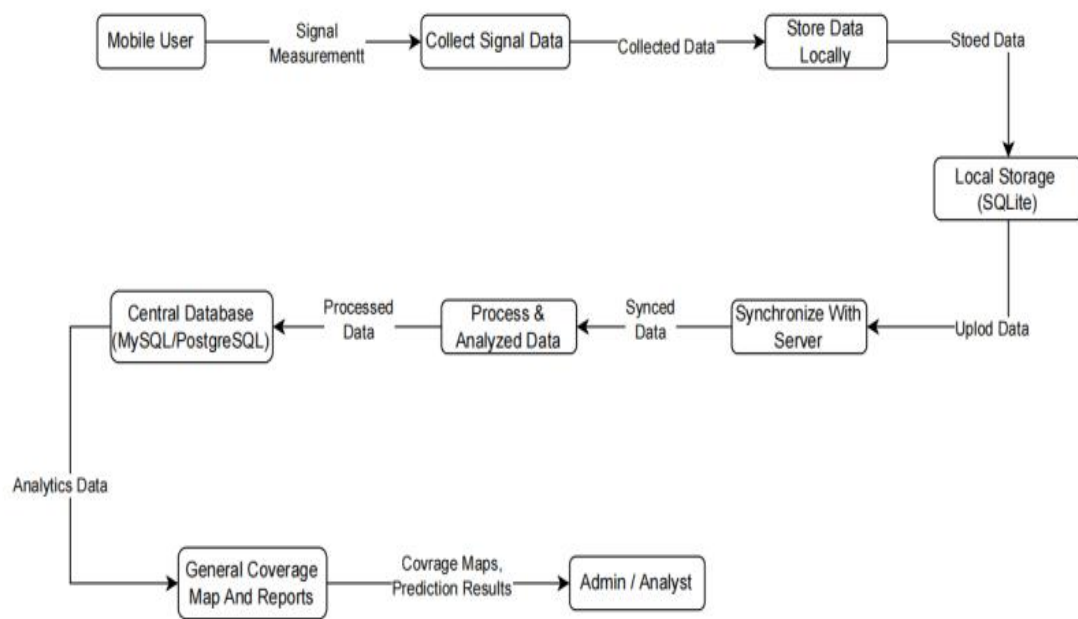
The overall system boundary interacts with two explicit external entities: the **Mobile User** (who feeds raw signal, GPS, and device info into the system) and the **Admin / Network Analyst** (who sends configuration updates and extracts analytical coverage charts).



Level 1: Core Process Decomposition

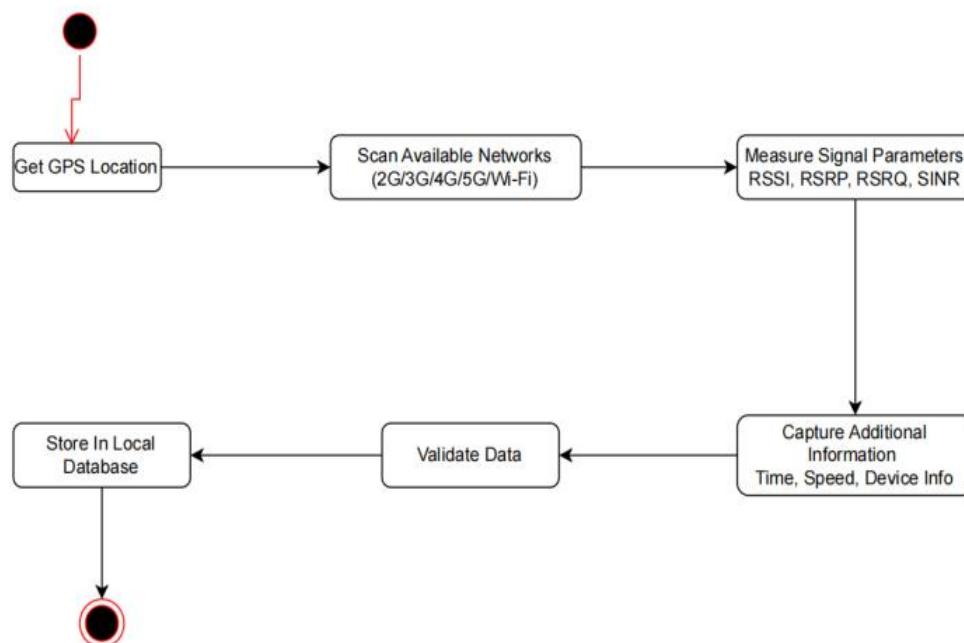
The application decomposes internal operations into five key sequential processes:

- **Collect Signal Data:** Captures instantaneous radio frequency metrics.
- **Store Data Locally:** Writes un-synchronized data directly to the device storage cache.
- **Synchronize Data with Server:** Pushes records to the cloud over a secure HTTPS channel upon network availability.
- **Process and Analyze Data:** Standardizes, cleans, and runs predictive analytics on incoming streams.
- **Generate Reports and Coverage Maps:** Aggregates telemetry into visible heatmaps.



Level 2: Detailed Breakdown of Signal Collection

Focuses tightly on Process 1, detailing the granular step-by-step actions executed on the device: retrieving GPS coordinates, scanning network bands extracting explicit signal parameters appending metadata, and performing client-side data validation.

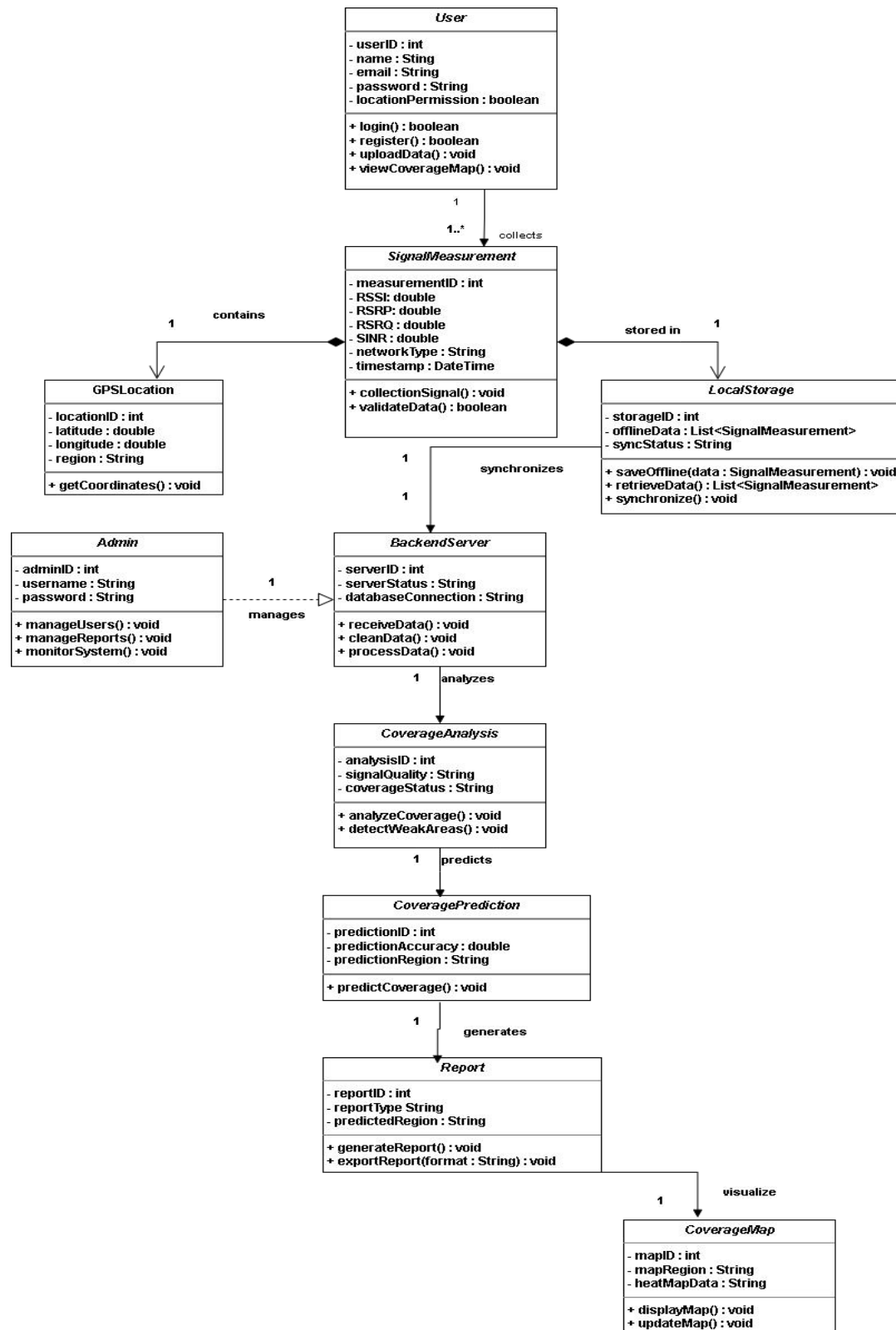


3.3.2 Structural Design (Class Diagram)

The internal structure of the software utilizes an object-oriented paradigm. The core domain entities are structured as follows:

User: Manages application runtime preferences and local permissions.

- ❖ **SignalMeasurement:** Encapsulates explicit radio frequency values mapped to a distinct GPSLocation.
- ❖ **LocalStorage:** Controls client-side buffering methods (saveOffline, synchronize).
- ❖ **BackendServer:** Coordinates structural ingestion, verification routines, and analytical workflows.
- ❖ **CoverageAnalysis & CoveragePrediction:** Implement algorithmic modeling to classify signal states and extrapolate grid values.

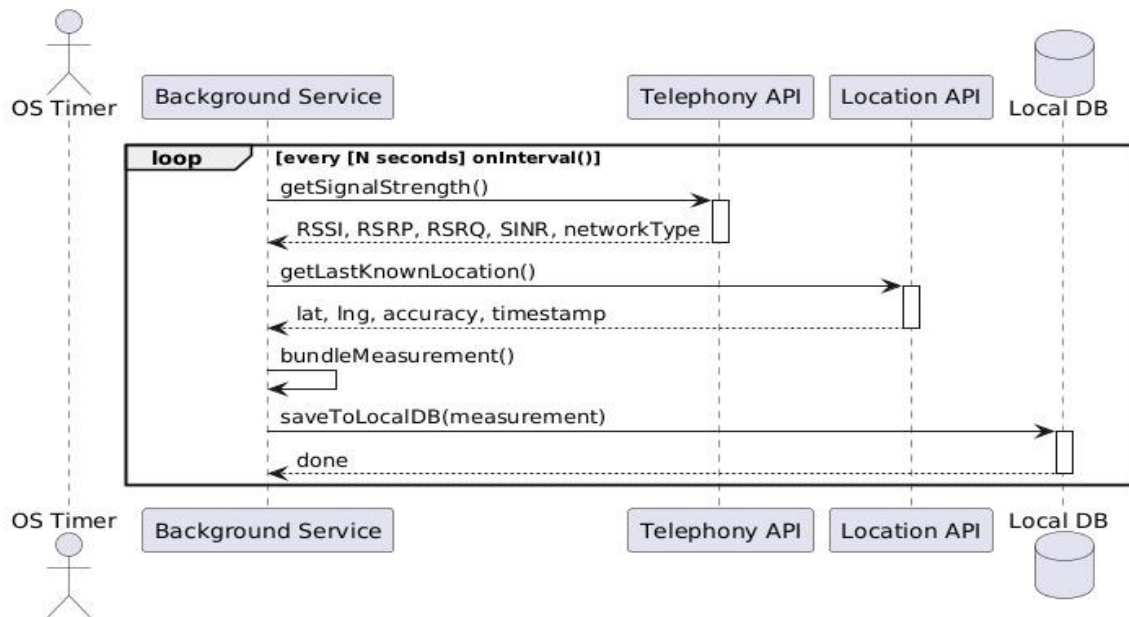


3.3.3 Behavioral Sequence Design

The system logic is built around an asynchronous design pattern to preserve UI performance during heavy background operations.

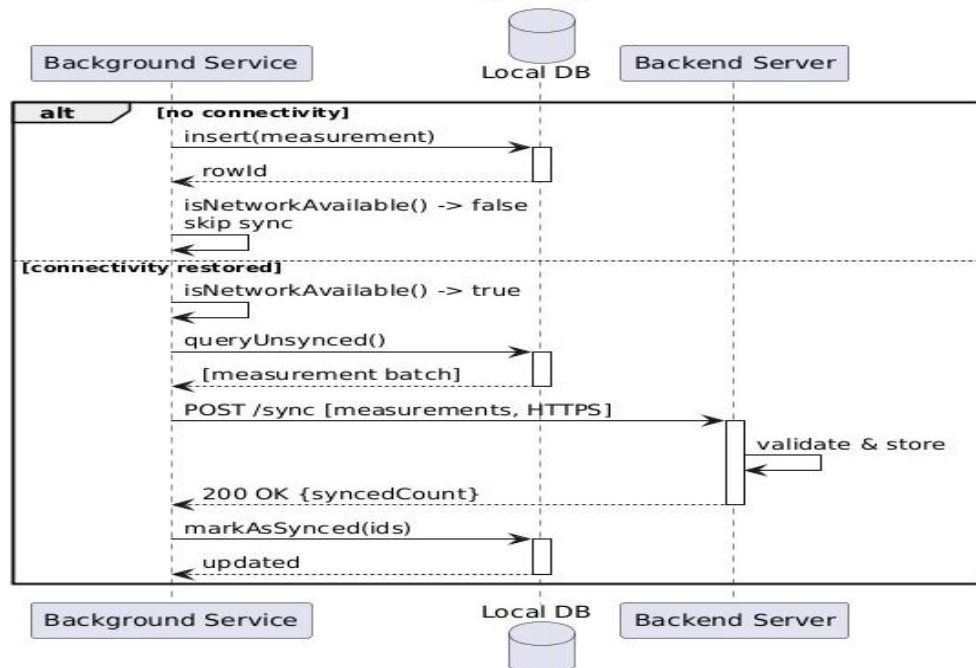
- **Background Sensing Loop:** Activated via an OS Timer every seconds, calling the platform's Telephony and Location APIs silently without interrupting user tasks.

SD1 - Background Signal Data Collection



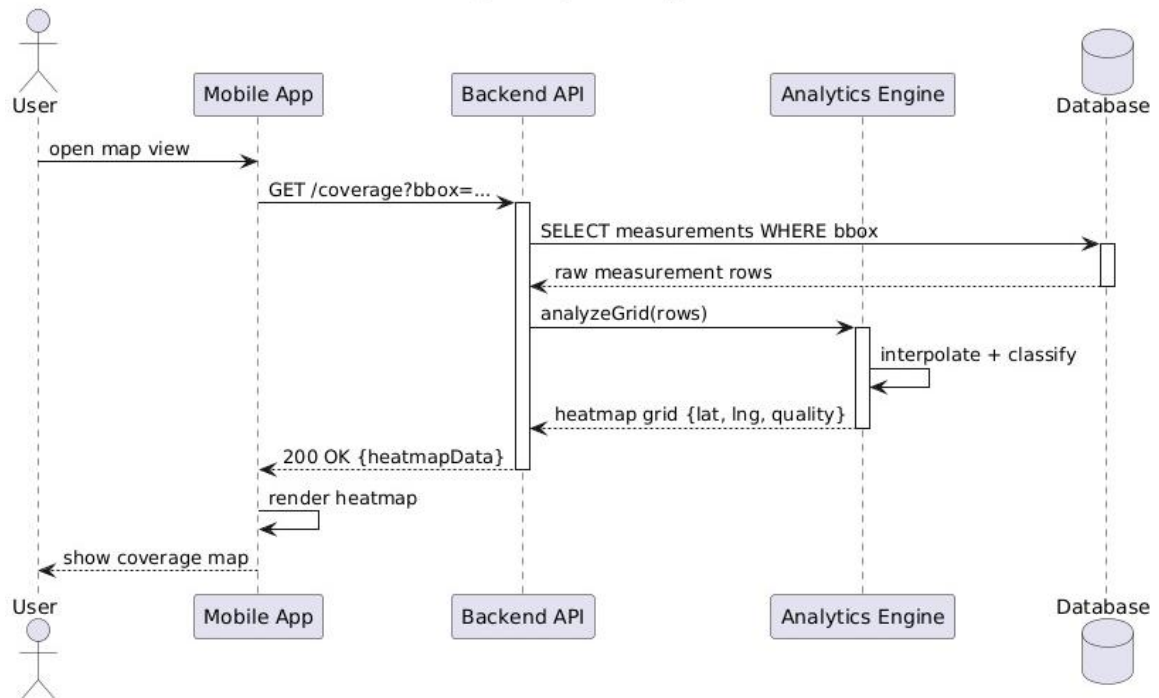
- **Offline Buffering State:** If internet connectivity drops (`isNetworkAvailable() -> false`), the background worker catches the exception and routes the payload to the local database container.

SD2 - Offline Storage & Synchronization

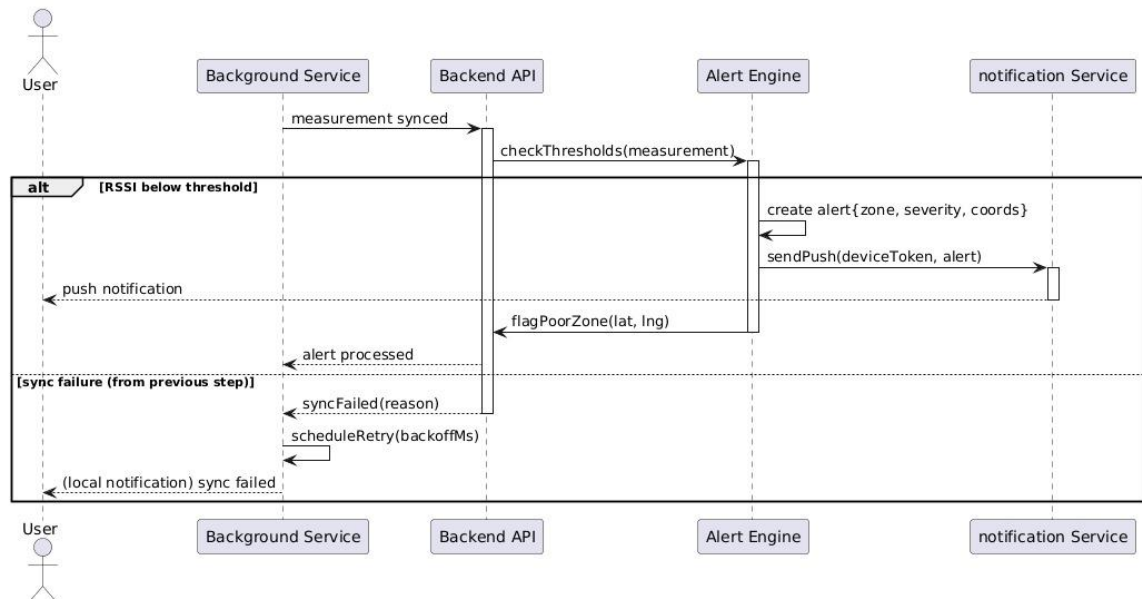


- **Active Cloud Sync State:** Upon restoration of a network connection, the application queries un-synchronized batches and transmits them via HTTP POST to the backend API, marking them as synced upon a successful 200 OK server response.

SD4 - Coverage Analysis & Map Visualization

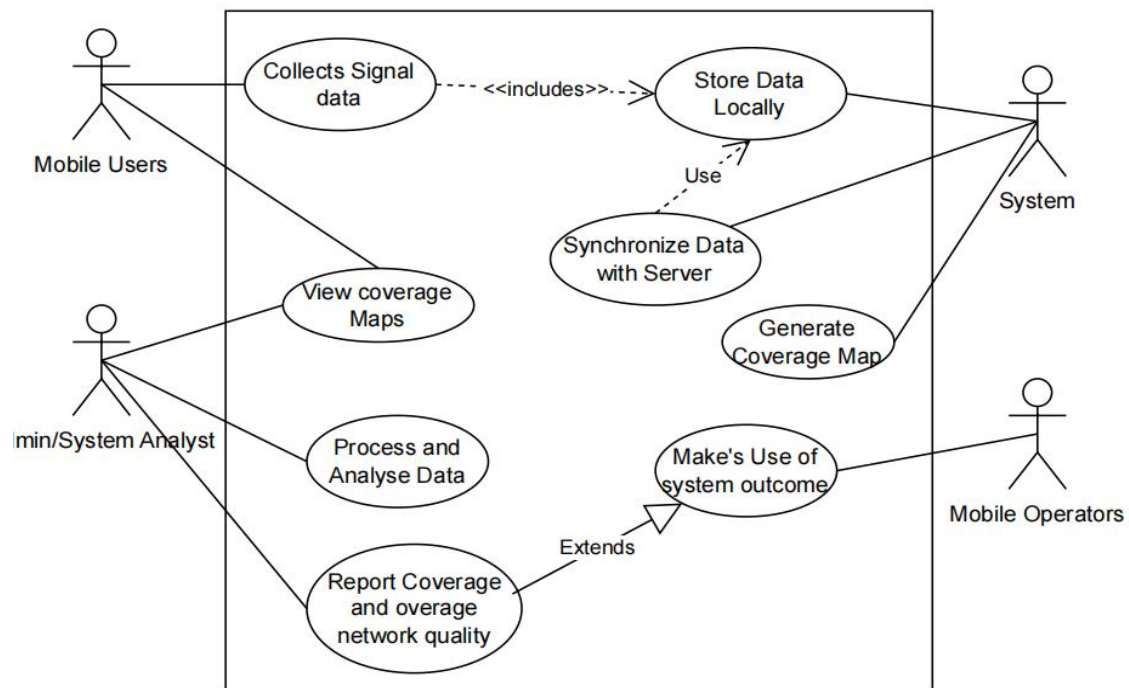


SD5 - Notifications & Alerts



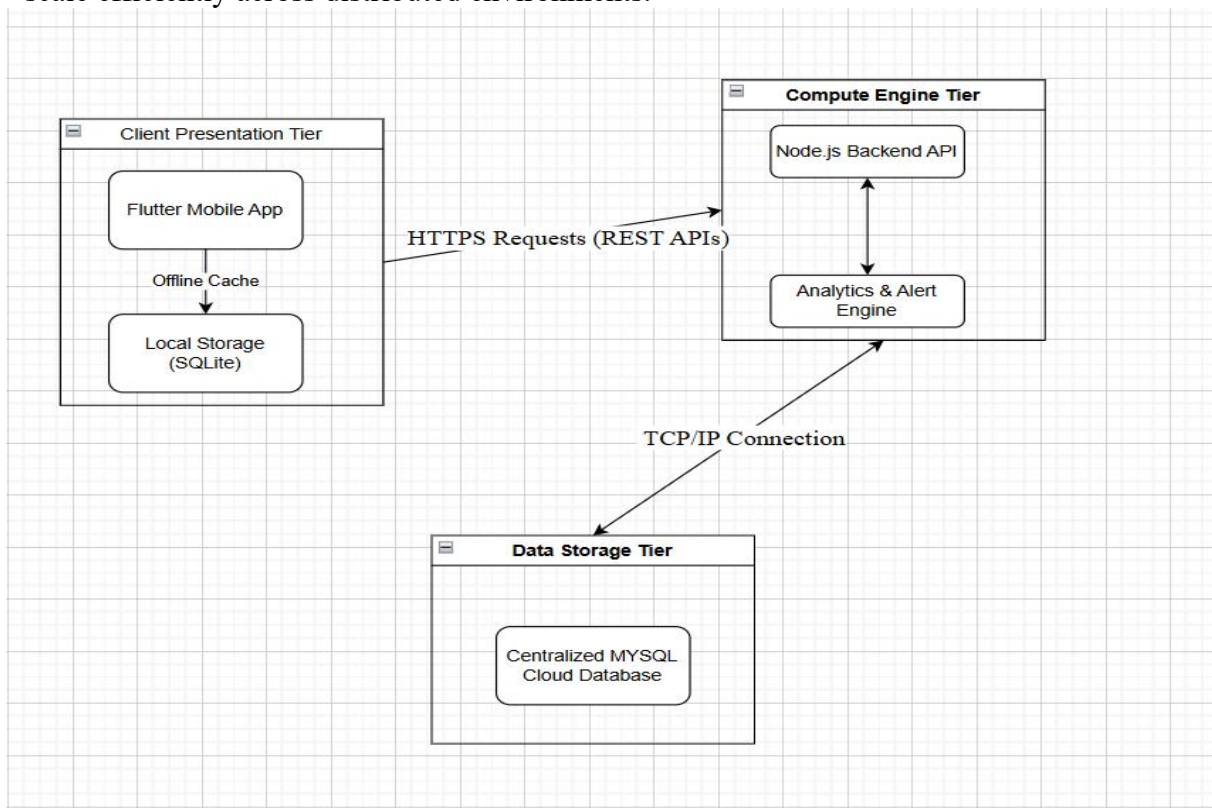
3.3.4 Use Case Specifications

The functional scope is governed by explicit actors mapping to system capabilities:



3.4. Global Architecture of the Solution

The physical implementation follows a multi-tier, decoupled architecture designed to scale efficiently across distributed environments.



- **Client Tier:** A cross-platform **Flutter & Dart** application executing on the user's smartphone. It manages real-time sensor loops and maintains an independent, lightweight **SQLite** instance to serve as an offline buffer.
- **Application Services Tier:** A fast, asynchronous **Node.js Rest API** environment that processes concurrent data streams, manages server-side validation routines, and hosts the underlying analytics and alert engines.
- **Persistent Storage Tier:** A centralized **MySQL Relational Database** cloud deployment configured with constraints, unique indexing, and relations designed to safeguard data integrity.

3.5. Description of the Resolution Process

The operational workflow transforms raw, distributed mobile sensor telemetry into centralized network intelligence through a six-step pipeline:

- ✓ **Sensing:** Automatically captures RSRP/RSRQ/SINR & GPS metrics via background hooks.
- ✓ **Caching:** Securely saves records to local SQLite database if offline.
- ✓ **Transfer:** Synchronizes cached records in batches via HTTPS to Node.js REST endpoints.
- ✓ **Ingestion:** Validates payload integrity and commits records to MySQL persistent tables.
- ✓ **Analysis:** Classifies coverage blocks (Good/Acceptable/Poor) and maps coverage holes.
- ✓ **Reporting:** Renders dynamic heatmaps on user UIs and generates analyst diagnostics.

3.6. Partial Conclusion

The analysis and design framework presented in this chapter provides a clear architectural blueprint for **SignalSense**. By establishing clear boundaries, modeling data transitions, and defining a decoupled, offline-first deployment architecture, the system is well-positioned for practical execution. The structural logic and relationships defined here transition directly into the physical implementation phase, which details database configuration, API routing, and frontend dashboard construction.

CHAPTER 4: DATABASE DESIGN AND BACKEND IMPLEMENTATION

4.1 Introduction

The storage layer and application middleware represent the structural foundation of the **SignalSense** platform. While the frontend mobile clients autonomously gather telemetry parameters at the network edge, the backend ecosystem handles batch data ingestion, parameter validation, schema integration, and high-performance querying.

This chapter presents the design and implementation of the project's data storage tier and backend logic. It traces the transition from conceptual object models to concrete relational database structures, specifies data integrity constraints, provides detailed tables mapping, and presents structural layouts for server-side endpoints and synchronization controls.

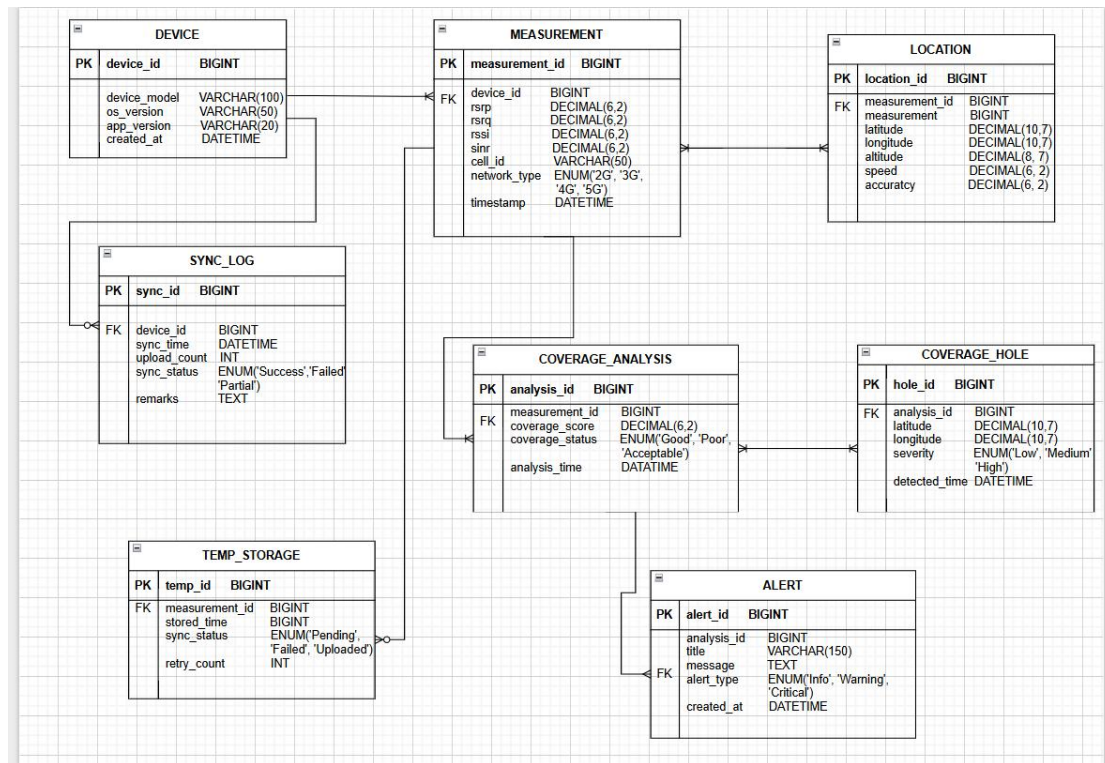
4.2 Database Architecture and Conceptual Design

The system uses a decoupled, hybrid database approach designed to balance reliability with high spatial performance. Mobile devices use an embedded **SQLite** storage engine to act as a resilient local cache when offline. Once connectivity is restored, the cloud tier uses a high-availability MySQL relational database engine to manage historical indexing and multi-dimensional spatial calculations.

4.2.1 Entity Relationship Diagram (ERD) Breakdown

To safeguard user privacy and encourage participation, the database structure intentionally avoids account registration barriers, maintaining user anonymity. The schema links physical network telemetry directly to geographic coordinates and infrastructure components across eight primary tables:

- ❖ **Locations:** Stores the exact spatio-temporal coordinate data point for each sensor read.
- ❖ **CellTowers:** Tracks infrastructure parameters to map signal telemetry back to physical carrier infrastructure.
- ❖ **Measurements:** The central fact table linking a location point to a cell tower, storing core radio parameters.
- ❖ **NetworkProviders:** Maps operator codes to specific provider names (e.g., MTN, Orange, Camtel) for filtered analytics.
- ❖ **CoverageHoles:** Tracks dropped connections or areas with severe coverage issues for operator notifications.
- ❖ **SyncLogs:** Audits client synchronization batches to maintain data consistency.
- ❖ **DeviceMetadata:** Stores smartphone hardware models to account for different receiver performance profiles.
- ❖ **SystemConfigs:** Maintains global operational variables like dynamic collection intervals.



4.2.2 Data Dictionaries and Schema Configurations

The physical schema tables are defined below, optimized with standard data types, index mappings, and spatial constraints.

Table 4.1: Locations Table Schema

Column Name	Data Type	Constraints	Description
LocationID	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate identifier for the spatial entry.
Latitude	DECIMAL(10, 8)	NOT NULL	Exact geographic latitude coordinate.
Longitude	DECIMAL(11, 8)	NOT NULL	Exact geographic longitude coordinate.
Altitude	DECIMAL(6, 2)	NULLABLE	Device height above sea level in meters.
Speed	DECIMAL(4, 2)	NOT NULL, DEFAULT 0.0	Movement velocity vector in meters per second.
Accuracy	DECIMAL(4, 2)	NOT NULL	GPS horizontal dilution of precision radius.

Table 4.2: CellTowers Table Schema

Column Name	Data Type	Constraints	Description
CellTowerID	INT	PRIMARY KEY, AUTO_INCREMENT	Internal surrogate identifier for the sector.
CellID	BIGINT	NOT NULL, UNIQUE(CellID, TAC_LAC)	Global cell transceiver infrastructure key.
TAC_LAC	INT	NOT NULL	Tracking Area Code / Location Area Code.
MNC	INT	NOT NULL	Mobile Network Code (e.g., 01 for MTN Cameroon).
MCC	INT	NOT NULL	Mobile Country Code (e.g., 624 for Cameroon).
RadioType	VARCHAR(10)	NOT NULL	Network generation

Table 4.3: Measurements Table Schema

Column Name	Data Type	Constraints	Description
MeasurementID	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Main surrogate log entry index.
LocationID	BIGINT	FOREIGN KEY \$→\$ Locations	Links the measurement to its spatial coordinates.
CellTowerID	INT	FOREIGN KEY \$→\$ CellTowers	Links the measurement to the hosting tower sector.
RSRP	DECIMAL(5, 2)	NOT NULL	Reference Signal Received Power.
RSRQ	DECIMAL(4, 2)	NOT NULL	Reference Signal Received Quality .
SINR	DECIMAL(4, 2)	NOT NULL	Signal-to-Interference-plus-Noise Ratio.
RSSI	DECIMAL(5, 2)	NULLABLE	Wideband received signal strength value (\$dBm\$).
BatteryDrain	DECIMAL(4, 2)	NOT NULL	Battery

			consumption rate during the sample.
Timestamp	DATETIME	NOT NULL	Temporal log mark matching the hardware read.

4.3 Ingestion Pipeline Implementation

The application middleware layer is built inside a highly asynchronous **Node.js** runtime using the **Express** framework, creating a non-blocking ingestion channel capable of processing concurrent batch uploads from thousands of active mobile edge nodes.

4.3.1 Application Entry Script Structure

The entry point architecture instantiates the web application runtime environment, configures security headers using standard middleware, manages incoming request payload size limits, and exposes connection pool configurations to avoid database starvation during concurrent edge uploads.

```

15  const reportsDir = path.join(__dirname, 'reports');
16  if (!fs.existsSync(reportsDir)) {
17    fs.mkdirSync(reportsDir, { recursive: true });
18  }
19
20  const app = express();
21  const port = process.env.PORT || 5000;
22
23  app.use(cors());
24  app.use(express.json({ limit: '10mb' }));
25  app.use('/reports', express.static(reportsDir));
26
27  app.use('/api/v1', mobileRoutes);
28  app.use('/api/v1/admin', adminRoutes);
29
30  app.use((err, req, res, next) => {
31    console.error('Unhandled error:', err);
32    res.status(500).json({ error: 'Internal server error' });
33  });
34
35  app.listen(port, () => {
36    console.log(`SignalSense backend running on port ${port}`);
37  });
38

```

4.3.2 Sync Management Core Logic

The application synchronization layer processes the uploaded telemetry payload data vectors. It handles structured batch synchronization steps, enforces data dictionary rules, verifies geolocation data elements, and wraps all write tasks inside explicit database transactions to ensure consistency across the related database tables.


```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\PaulFleming\Desktop\All_Projects\signalsense_backend> curl http://localhost:5000/api/v1/health

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run
when the page is parsed.
RECOMMENDED ACTION:
  Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): y

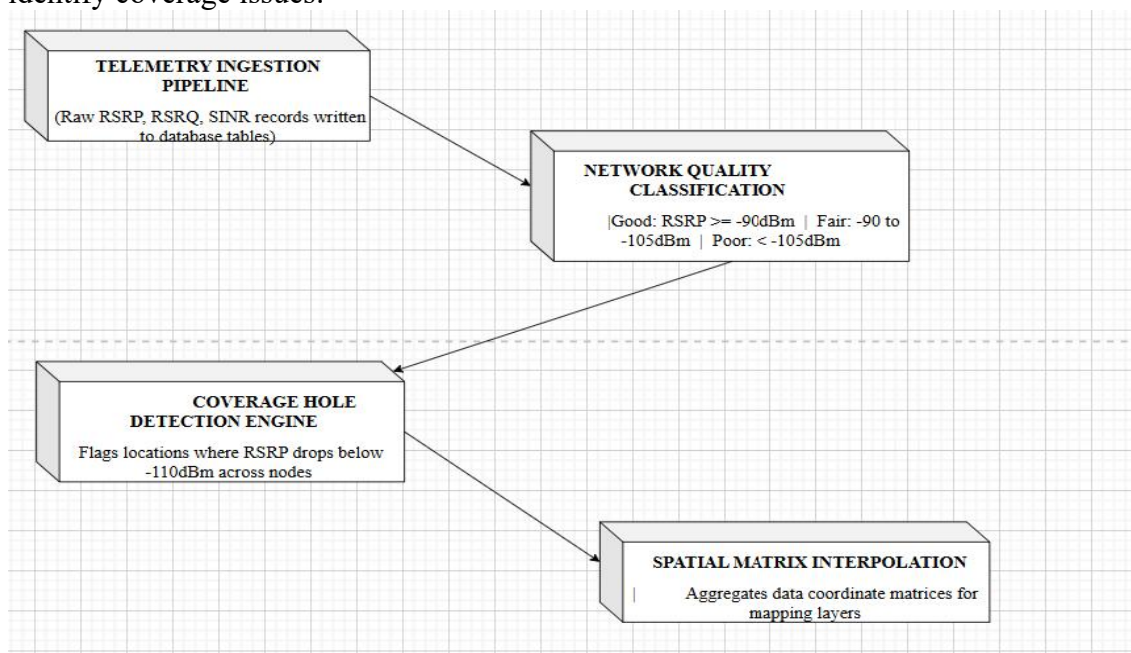
StatusCode      : 200
StatusDescription : OK
Content         : {"status":"ok"}
RawContent      : HTTP/1.1 200 OK
                  Access-Control-Allow-Origin: *
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 15
                  Content-Type: application/json; charset=utf-8
                  Date: Tue, 16 Jun 2026 12:47:20 GMT
                  ...
Forms           : {}
Headers         : {[Access-Control-Allow-Origin, *], [Connection, keep-alive], [Keep-Alive,
                  timeout=5], [Content-Length, 15]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 15

PS C:\Users\PaulFleming\Desktop\All_Projects\signalsense_backend> |

```

4.4 Data Quality Categorization and Analysis

Once raw measurement data is saved into persistent storage tables, the backend engine automatically runs analytics pipelines to categorize network performance and identify coverage issues:



1. Network Quality Classification: Measurements are parsed by backend evaluation routines that categorize network status based on key signal parameters:

- **Good:** RSRP > -90dBm (Strong signal field; high data throughput capacity).
- **Fair:** RSRP between -90dBm and -105dBm (Stable connection quality; standard operational baseline).
- **Poor:** RSRP < -105dBm (Degraded connectivity; high risk of service drops or call failures).

2. **Coverage Hole Detection Engine:** The system aggregates telemetry streams over spatial clusters. If a specific area consistently drops below the minimum threshold RSRP < -110dBm across multiple independent device nodes, the backend generates an alert entry in the 'Coverage Holes' index, pinpointing dead zones for operators.

3. **Spatial Matrix Interpolation:** The backend system indexes coordinate strings into dynamic spatial data matrices, generating clean geospatial outputs to render real-time coverage heatmaps on frontend dashboards.

5.5 Partial Conclusion

The database structures and ingestion logic detailed in this chapter provide a high-throughput backend architecture for **SignalSense**. Using a connection-pooled Node.js architecture paired with structured database constraints and transaction controls allows the system to securely process large volumes of crowdsensed telemetry records. This operational foundation ensures data reliability, providing a clean dataset for coverage visualization, automated hole detection, and network analysis.

CHAPTER 5. CONCLUSION AND FURTHER WORKS

1. Summary of Findings

The development, deployment, and testing of the **SignalSense** crowdsensed drive test mobile application have yielded critical insights into the viability of decentralized network quality monitoring. This project successfully demonstrated that everyday consumer smartphones can function effectively as autonomous sensing nodes to map cellular network coverage and diagnose performance drops. The core findings from the project implementation are summarized as follows:

- ✓ **Viability of Mobile Crowdsensing (MCS):** The physical-layer data collection loops running on edge devices proved that high-fidelity cellular metrics can be captured passively without requiring specialized telemetry hardware or manual drive testing vehicles.
- ✓ **Resilience of Offline-First Caching:** The multi-tier architecture using a client-side **SQLite** cache container successfully addressed transient cellular network dropouts. Telemetry data gathered within severe coverage holes was reliably buffered locally and automatically synchronized with the cloud backend once an internet connection was re-established.
- ✓ **Battery and Resource Trade-offs:** Initial testing highlighted a direct relationship between sampling frequency and device resource consumption. By optimizing background workers to run on a 30-second adaptive interval, the system minimized background battery drain, ensuring it remained light enough for continuous user operation.
- ✓ **Geospatial and Analytical Insights:** When processed through the centralized **Node.js** and **MySQL** infrastructure, the aggregated telemetry data enabled the automatic classification of network health and identified spatial boundaries for regional network dead zones.

2. Contribution to Engineering and Technology

The design and implementation of **SignalSense** provides distinct technical advancements over existing commercial and academic platforms. While commercial tools like **OpenSignal** and **nPerf** deliver general coverage indicators, they operate under functional limitations that this project directly addresses.

The primary engineered differentiators include:

- ❖ **Privacy-First, Registration-Free Architecture:** Unlike commercial tools that mandate account registration, link emails, or log tracking markers—creating high entry barriers—**SignalSense** enforces an anonymous data collection path. It strips away identity tracking at the edge, logging purely physical-layer telemetry linked to structural coordinates to ensure user privacy.
- ❖ **Edge-Validated Batch Synchronization Over Weak Channels:** Standard network tools frequently lose unsynchronized metrics when server uploads fail during complete signal drops. The edge pipeline designed for **SignalSense** uses transactional state logging inside a local **SQLite** database, allowing the client to batch payload strings and securely push data out only after a valid 200 OK server response is received.

- ❖ **Decoupled, Relational Open Schema Design:** Existing commercial solutions rely on proprietary, closed-source processing systems, hiding raw historical logs from regional operators and researchers. SignalSense introduces an open, relational schema mapping spatial coordinates, cell sector identifiers, and radio metrics into optimized MySQL tables. This open structure provides accessible, granular data datasets necessary for targeted network planning and localized engineering deep-dives.

3. Recommendations

To maximize the operational effectiveness of crowdsensed cellular tracking platforms like SignalSense, the following strategies are highly recommended for telecommunication regulatory authorities and mobile network operators:

- ❖ **Implementation of Incentive Programs:** To build a dense, high-volume grid of telemetry points, operators should provide passive data contributors with micro-incentives, such as promotional data bundles or SMS credits. This strategy helps maintain a consistent stream of active background nodes.
- ❖ **Integration with Regulatory Quality of Service (QoS) Audits:** Telecommunication regulatory bodies should adopt decentralized crowdsourced architectures alongside traditional drive testing. This approach provides continuous, real-time metrics that reflect true end-user network quality inside homes and commercial buildings.
- ❖ **Dynamic Client Profiling:** The backend analytical tier should include hardware-specific receiver correction weights. Because radio frequency receiver sensitivities vary across different smartphone manufacturers, adjusting for these variations helps ensure uniform data interpretation across diverse device ecosystems.

4. Difficulties Encountered

Developing an automated network measurement system that operates across varied mobile environments presented several technical and logistical challenges:

- ❖ **Strict Operating System Energy Restrictions:** Modern mobile operating systems (Android and iOS) enforce strict background execution rules to maximize device battery life. Bypassing aggressive OS thread limitations required configuring persistent foreground service wrappers and visible system tray notifications to prevent the operating system from terminating the background sensing loops.
- ❖ **API Heterogeneity and Vendor Fragmentation:** Gathering uniform hardware measurements proved difficult due to differences in how hardware manufacturers implement Android's TelephonyManager API. Certain devices omitted critical parameters like SINR or returned static fallback values, requiring the development of robust exception-handling routines to maintain clean data ingestion.
- ❖ **GPS Power Disruption and Spatial Divergence:** Continuous, high-accuracy GPS tracking rapidly drains smartphone batteries. The development team encountered significant battery drain during high-frequency sampling intervals, which required moving to an adaptive location loop that adjusts sampling rates based on device speed and activity states.

5. Further Works

While the current version of the SignalSense application establishes a reliable framework for telemetry collection, caching, and mapping, several key areas have been identified for future research and technical enhancement:

Advanced Spatial Interpolation via Machine Learning: Future updates should replace standard coordinate grid mapping with machine learning models (such as *Kriging* or *Deep Neural Networks*). Implementing these algorithms will allow the